

Исчерпывающий анализ архитектуры ADT-файлов и алгоритмов извлечения топологии дорожных сетей в World of Warcraft 3.3.5a

Глубинная реконструкция внутриигровых навигационных графов на основе сырых бинарных данных клиента World of Warcraft (версия 3.3.5a, билд 12340, Wrath of the Lich King) представляет собой масштабную задачу в области реверс-инжиниринга и вычислительной геометрии. В отличие от современных игровых движков, где дороги зачастую реализуются в виде независимых векторных сплайнов или отдельных полигональных мешей, движок World of Warcraft полагается на парадигму текстурного блендинга (texture splatting). В этой системе дорожное полотно не существует как геометрический объект; оно является лишь одним из нескольких текстурных слоев, накладываемых поверх базового рельефа с использованием интерполированных масок прозрачности (Alpha Maps).¹

Настоящий отчет предоставляет исчерпывающий структурный, математический и алгоритмический анализ методов извлечения этих неявных дорожных сетей из клиентских архивов MPQ. Исследование охватывает спецификации бинарного формата ADT (A Dungeon Terrain), особенности парсинга чанков MCNK, MCLY и MCAL, математический аппарат пространственного отображения (маппинга) пикселей на глобальные координаты, а также применение методов компьютерного зрения (скелетонизации) для трансформации растровых масок в связные навигационные графы.

Фундаментальная архитектура ландшафтной системы и структура MPQ/ADT

Ландшафтные данные World of Warcraft инкапсулированы в проприетарные архивы MPQ (MoPaQ), управляемые библиотекой StormLib. Для извлечения данных требуется чтение архивов common.MPQ, expansion.MPQ, lichking.MPQ и соответствующих патчей, где файлы организованы в иерархическую виртуальную файловую систему.²

Основой организации открытого мира является деление глобальной карты (континента) на регулярную сетку размером 64×64 тайла. Каждый такой тайл представлен отдельным файлом ADT (например, World\Maps\Azeroth\Azeroth_32_32.adt) и покрывает физическую площадь ровно $533.33333 \times 533.33333$ внутриигровых ярдов.¹ Важно отметить, что в до-катаклизмовскую эпоху (версии до 4.x.x) файлы ADT были монолитными⁵; разделение на суффиксы _tex0.adt и _obj0.adt произошло только в последующих дополнениях.¹ Таким

образом, для WotLK 3.3.5a вся геометрия, текстуры и альфа-карты хранятся в одном файле.

Система бинарных чанков и проблема обратных тегов (Reversed Tags)

Формат ADT базируется на иерархической системе бинарных чанков, заимствованной из формата RIFF. Каждый чанк начинается с 8-байтового заголовка: 4 байта идентификатора (FourCC) и 4 байта размера полезной нагрузки (uint32). При программной реализации парсера на языке Python (и других языках, работающих на архитектуре x86/x64) возникает критический нюанс, связанный с порядком байтов (Endianness).

Поскольку архитектура x86 использует порядок little-endian (от младшего к старшему), 32-битное целое число, представляющее собой ASCII-строку MCNK (Map Chunk), при побайтовом чтении в память и последующей интерпретации будет перевернуто. Следовательно, в гексадецимальном редакторе или при чтении файла тег MCNK фактически записан и читается алгоритмами распаковки как KNCM.¹ Аналогично, MCAL читается как LACM, а MCLY как YLCM. Любой парсер должен либо инвертировать прочитанные строки, либо использовать константы с обратным порядком символов для поиска и валидации чанков.

Глобальная система координат

Движок WoW использует правостороннюю декартову систему координат, которая значительно отличается от стандартов, принятых в современных ГИС и 3D-редакторах. В этой системе:

- Ось $X+$ направлена строго на Север.
- Ось $Y+$ направлена строго на Запад.
- Ось $Z+$ обозначает вертикальную высоту (up vector), где 0 соответствует уровню моря.¹

Начало координат $(0, 0, 0)$ располагается точно в центре глобальной карты континента.¹ Вследствие этого, координаты левого верхнего (Северо-Западного) угла карты принимают максимальные положительные значения $(17066.6666, 17066.6666)$, а правого нижнего (Юго-Восточного) — минимальные отрицательные $(-17066.6666, -17066.6666)$.¹

Формула трансформации глобальных мировых координат (X_{wow}, Y_{wow}) в индексы тайла ADT $(TileX, TileY)$ отражает эту инвертированную логику:

$$TileX = 31 - \lfloor \frac{X_{wow}}{533.33333} \rfloor$$

$$TileY = 31 - \lfloor \frac{Y_{wow}}{533.33333} \rfloor$$

Внутренняя топология ADT: Навигация по чанкам MCNK и MCIN

Внутри каждого ADT-файла ландшафт детализируется посредством разбиения основного тайла на 256 квадратных сегментов (сетка 16×16). Эти сегменты представлены чанками MCNK. Каждый MCNK покрывает физическую площадь 33.33333 × 33.33333 ярда.¹

Для обеспечения константного времени доступа к произвольному участку ландшафта (что критически важно для производительности клиента игры и утилит реверс-инжиниринга), ADT-файл содержит индексный чанк MCIN (Map Chunk INdex). MCIN всегда располагается в начале файла после главного заголовка MHDR и содержит ровно 256 записей по 16 байт каждая.⁷

Поле в структуре MCIN	Тип данных	Описание
offset	uint32	Смещение до начала чанка MCNK
size	uint32	Полный размер чанка MCNK, включая данные
flags	uint32	Флаги состояния (обычно 0x1 - loaded)
asynclid	uint32	Идентификатор асинхронной загрузки (pad)

Важнейший технический нюанс парсинга заключается в природе этих смещений. Смещения, указанные в чанке MCIN, являются **абсолютными** по отношению к началу самого файла ADT.¹ Это позволяет парсеру выполнять seek() напрямую к требуемому участку геометрии без необходимости последовательно считывать все предыдущие блоки данных.

Структура заголовка MCNK и относительные смещения

Каждый чанк MCNK имеет собственный сложный заголовок размером 128 байт, за которым следуют вложенные под-чанки, описывающие геометрию высот (MCVT), нормали (MCNR), конфигурацию слоев текстур (MCLY) и их альфа-карты (MCAL).¹

В заголовке MCNK хранятся указатели на эти вложенные структуры. В отличие от MCIN, смещения ofsHeight, ofsNormal, ofsLayer и ofsAlpha являются **относительными смещениями от начала чанка MCNK**, то есть от байта, где начинается магическая последовательность MCNK (или KNCM в памяти), а не от конца заголовка.¹

Например, чтобы прочитать данные альфа-карт, алгоритм должен вычислить физическую позицию в файле по формуле:

$$Position_{MCAL} = Offset_{MCNK_Absolute} + MCNK.ofsAlpha$$

Кроме того, по смещению 0x68 в заголовке MCNK хранится массив из трех чисел с плавающей запятой (float), описывающий базовую пространственную позицию данного MCNK-чанка. Координаты хранятся в специфическом порядке:

$(wowY, wowZ, wowX)$.¹ Этот вектор определяет пространственное положение Северо-Западного угла чанка и служит отправной точкой для маппинга альфа-пикселей на мировые координаты.

Идентификация дорожных покрытий: Семантический анализ текстур (MTEX)

Для того чтобы извлечь дорожную сеть, утилита должна сначала определить, какие именно участки ландшафта покрыты дорожными материалами. Эта информация не хранится в виде готового векторного пути; вместо этого дороги "нарисованы" на ландшафте.

Чанк MTEX (Map Textures) в структуре ADT содержит непрерывный блок памяти, в котором друг за другом записаны нуль-терминированные ASCII-строки.¹ Эти строки представляют собой полные абсолютные пути к файлам текстур в формате BLP (Blizzard Map Format) внутри MPQ-архивов.¹ Индексы этих строк (начиная с 0) используются в чанках MCLY для указания, какая текстура применена к конкретному слою MCNK.

Паттерны именования дорожных текстур по локациям

Анализ файлов различных локаций (Элвинский лес, Западный Край, Дун Морог) демонстрирует, что разработчики Blizzard использовали стандартизированную номенклатуру для дорожных покрытий, однако она жестко привязана к биому.⁹

Локация (MapID)	Биом	Типичные текстуры дорог (BLP)	Паттерны для иных поверхностей
Elwynn Forest (0)	Лес, умеренный климат	Tileset\Elwynn\ElwynnDirt01.blp Tileset\Elwynn\ElwynnCobblestone01.blp Tileset\Elwynn\ElwynnPath.blp	ElwynnGrass01.blp ElwynnRock.blp
Westfall (0)	Засушливые прерии	Tileset\Westfall\WestfallDirt.blp Tileset\Westfall\WestfallDust.blp	WestfallGrass.blp WestfallSand.blp
Dun Morogh (0)	Заснеженные горы	Tileset\DunMorogh\DunMoroghSnowDirt.blp Tileset\DunMorogh\DunMoroghSnowTrack.blp	DunMoroghSnow.blp DunMoroghIce.blp

Как видно из примеров, текстурные пути часто содержат лексемы **Dirt**, **Cobblestone**, **Path** или **Track**. Однако использование исключительно лексического анализа (поиска подстроки "Road" или "Dirt") в имени файла является ненадежной стратегией для классификации. Текстура ElwynnDirt01.blp может применяться не только для формирования непрерывного дорожного тракта, но и для визуализации грязи вокруг фермерских угодий (например, Stonefield Farm), крутых грунтовых склонов водоемов или просто для внесения визуального разнообразия в однородный травяной покров.⁹

Требуется более детерминированный механизм, позволяющий отличить "грязь как дорогу" от "грязи как декоративного пятна". Этот механизм скрыт в метаданных клиентской базы данных.

Стратегия классификации: Симбиоз MCLY и

GroundEffectTexture.dbc

Чанк MCLY внутри каждого MCNK содержит до 4 структур (по 16 байт каждая), описывающих конфигурацию наложенных текстурных слоев.¹

Структура записи MCLY в версии 3.3.5a:

- uint32 textureId — индекс строки в чанке MTEX.
- uint32 flags — битовая маска параметров рендеринга и анимации.
- uint32 offsetInMCAL — смещение данных прозрачности для данного слоя относительно начала чанка MCAL.
- int32 effectId — идентификатор визуальных и физических эффектов.¹

Поле effectId является внешним ключом к таблице GroundEffectTexture.dbc в клиентских данных. Именно этот файл содержит исчерпывающую информацию о семантике поверхности, и анализ его полей дает стопроцентную точность в идентификации дорог.¹¹

Структура GroundEffectTexture.dbc

Таблица связывает текстуру с процедурно генерируемыми мелкими объектами (Doodads), такими как пучки травы, мелкие камни или цветы, а также определяет звуковые эффекты при ходьбе персонажа.¹¹

Колонка	Описание	Применение для идентификации дорог
ID	Уникальный ключ, соответствующий effectId из MCLY	Базовый маппинг.
Doodad Array	Ссылки на модели (трава, цветы). Значение -1 означает отсутствие объекта.	Ключевой маркер. Дороги (Roads) всегда имеют значение -1, чтобы на них не генерировалась трава.
Density	Плотность генерации Doodad объектов.	На дорогах игнорируется, так как массив Doodad содержит -1.
Terrain Type / Sound	Идентификатор звука шагов (грязь, камень, снег)	Подтверждает, что поверхность является

	и системы частиц (брызги грязи, пыль).	твердой/утоптанной (footstep_dirt, footstep_stone).
--	--	---

Алгоритм высокоточной классификации дороги:

1. Извлечь effectId анализируемого слоя из MCLY.
2. Найти соответствующую запись в GroundEffectTexture.dbc.
3. Проверить массив Doodad Array. Если все 4 значения равны -1 (или эквивалентному пустому значению), это означает, что движок преднамеренно запрещает генерацию процедурной флоры на данном слое.¹¹
4. Проверить Terrain Type. Если он соответствует типам Dirt, Cobblestone, Wood, Stone или специфическим Snow_Track, вероятность того, что слой представляет собой дорогу, стремится к 100%.
5. Именно этот метод используют такие проекты, как Noggit (комьюнити-редактор карт) и сторонние Map Viewers, чтобы корректно отображать дороги без прорастающей сквозь них травы.¹¹

Флаги слоя (MCLY flags)

Битовые флаги MCLY не содержат явного атрибута "is_road". Однако они критичны для процесса извлечения:

- 0x100 (alpha_map): Указывает, что данный слой имеет карту прозрачности.
- 0x200 (alpha_map_compressed): Указывает, что альфа-карта сжата проприетарным RLE-алгоритмом.¹

Фундаментальное правило движка World of Warcraft: **Слой с индексом 0 (Layer 0) является базовым слоем (base texture) и никогда не имеет альфа-карты.**¹ Он всегда покрывает 100% поверхности чанка MCNK с абсолютной непрозрачностью. Дорожные текстуры, будучи наложенными поверх базового ландшафта, никогда не выступают в роли нулевого слоя. Дорога всегда располагается на слоях 1, 2 или 3, и, следовательно, всегда сопровождается ассоциированной матрицей в чанке MCAL.¹

Детализация формата MCAL (Alpha Maps) и алгоритмы декомпрессии

Чанк MCAL (Map Chunk Alpha Layers) содержит пространственные данные о прозрачности слоев текстур, заданных в MCLY. Эти данные представляют собой сетку размером 64×64 пикселя, где каждый пиксель определяет коэффициент смешивания (blending weight) текстуры в данной точке пространства.¹

Сложность извлечения заключается в том, что формат данных MCAL динамически варьируется. В WotLK (3.3.5a) существует три различных варианта кодирования, выбор которых детерминируется комбинацией флагов в MCLY (уровень слоя) и в глобальном заголовке MHDR / MPHD (уровень карты).¹

Вариант 1: Несжатый 4-битный формат (2048 байт)

Это исторический формат по умолчанию, унаследованный еще со времен Vanilla WoW. Он используется, если не установлены флаги 0x200 в MCLY и отсутствуют специальные флаги высокой детализации в MPHD.¹ В этом формате 4096 пикселей (64×64) упаковываются в 2048 байт. Каждый байт содержит два 4-битных значения прозрачности (от 0 до 15). Алгоритмическая распаковка (псевдокод Python):

Python

```
alpha_map = np.zeros((64, 64), dtype=np.uint8)
idx = 0
for byte in raw_mcal_data[0:2048]:
    # Младшие 4 бита - первый пиксель
    p1 = byte & 0x0F
    # Старшие 4 бита - второй пиксель
    p2 = (byte >> 4) & 0x0F

    # Нормализация до диапазона 0-255
    alpha_map.flat[idx] = (p1 * 17)    # 0x0F * 17 = 255
    alpha_map.flat[idx+1] = (p2 * 17)
    idx += 2
```

Вариант 2: Несжатый 8-битный формат (4096 байт)

Также известен как "Big Alpha". Активируется, если в чанке MPHD (находящемся в файле WDT, описывающем всю карту) установлены флаги 0x4 (adt_has_big_alpha) или 0x80 (adt_has_height_texturing).¹ В этом режиме каждый пиксель представлен полноценным 8-битным байтом (от 0 до 255), обеспечивая высокую плавность градиентов без необходимости умножения. Размер данных слоя в MCAL составляет ровно 4096 байт.¹

Вариант 3: Сжатый формат (Compressed RLE)

Сжатие активируется, если для конкретного слоя в MCLY установлен бит 0x200 (alpha_map_compressed).¹ Важно понимать, что базовая битовая глубина при этом

по-прежнему зависит от флагов MPHD.

Алгоритм сжатия представляет собой специализированную вариацию Run-Length Encoding (RLE).¹⁵ Поток данных состоит из управляющих байтов (токенов) и байтов данных. Старший бит управляющего байта определяет режим декодирования (копирование серии уникальных значений или дублирование одного значения). Поскольку размер сжатого блока MCAL не фиксирован, для безопасной итерации по слоям парсер должен использовать поле `offsetInMCAL` из MCLY. Оно содержит точное относительное смещение начала данных для конкретного слоя от начала данных чанка MCAL.¹ Это позволяет декодировать только нужный (дорожный) слой, пропуская остальные без необходимости их полной RLE-декомпрессии.

Математическое отображение (Mapping) альфа-карт на мировые координаты

После извлечения и нормализации сетки 64×64 MCAL, необходимо точно спроецировать каждый пиксель маски на глобальные 3D-координаты мира WoW, чтобы результирующий граф дороги совпадал с топологией ландшафта.

Матрица MCAL жестко привязана к границам одного MCNK чанка, физический размер которого составляет 33.33333 ярда. Следовательно, каждый пиксель альфа-карты покрывает площадь:

$$Size_{pixel} = \frac{33.33333}{64} \approx 0.520833 \text{ ярда}$$

Пространственная ориентация и интерполяция

Массив MCAL (64×64) не имеет перекрытий (overlap) между соседними чанками MCNK.¹ В отличие от карты высот MCVT, где сетки 9×9 и 8×8 переплетаются и делят общие вершины на границах, альфа-карты текстурируются встык. Нулевой столбец пикселей правого чанка вплотную примыкает к 63-му столбцу левого чанка.¹ При рендеринге в самом клиенте между центрами этих пикселей происходит билинейная интерполяция¹, создающая плавный переход, однако для целей извлечения графа достаточно работать с центрами дискретных пикселей.

Ориентация матрицы MCAL строго следует логике глобальной системы координат. Нулевой пиксель с индексами `` (первый байт декомпрессированных данных) математически привязан к **Северо-Западному (Top-Left)** углу MCNK чанка.¹ В декартовой системе WoW (ось X — Север, ось Y — Запад), продвижение по массиву MCAL (вправо по строке и вниз по столбцам) означает смещение на Юг и Восток.¹

Трансформационная матрица

Пусть $Base_X, Base_Y$ — мировые координаты Северо-Западного угла MCNK чанка. Эти значения напрямую считываются из заголовка MCNK (смещение 0x68, float position), где они записаны в порядке ``.¹

Координаты центра любого пикселя с индексами row (от 0 до 63) и col (от 0 до 63) в глобальной системе вычисляются по формулам:

$$World_X = Base_X - (row \times 0.520833) - \frac{0.520833}{2}$$

$$World_Y = Base_Y - (col \times 0.520833) - \frac{0.520833}{2}$$

Знак минус обусловлен тем, что увеличение индексов в массиве MCAL отдаляет точку от Северо-Западного угла (имеющего максимальные положительные значения осей X и Y) в сторону Юго-Востока (где значения осей X и Y уменьшаются). Вычитание половины размера пикселя смещает точку из угла пиксельной ячейки точно в ее геометрический центр, что повышает точность итогового графа.

Стратегия построения векторной дорожной сети: Алгоритмы компьютерного зрения

Извлечение тысяч локальных 64×64 масок для каждого MCNK и их трансляция в координаты формирует колоссальный бинарный растр для всего ADT-тайла (общее разрешение 1024×1024 пикселя). Следующий этап — трансформация этого растрового покрытия (где ширина дороги составляет от 10 до 20 пикселей, или 5-10 ярдов) в связный одномерный граф (nodes and edges), пригодный для навигации алгоритмом A* (A-Star).¹⁶

Бинаризация и выбор порога (Thresholding)

Значения в альфа-карте распределены в диапазоне ``. Из-за эффекта "feathering" (растущевки), применяемого левел-дизайнерами при "рисовании" дорог, края дорожного тракта имеют значения прозрачности от 10 до 100, плавно растворяясь в базовой текстуре травы.¹⁸ Включение этих полупрозрачных обочин в граф приведет к чрезмерному утолщению растра и появлению ложных ответвлений.

Эмпирически доказано, что оптимальный порог бинаризации лежит в диапазоне:

$$Mask_{binary} = \begin{cases} 1, & \text{если } MCAL_{val} > 128 \\ 0, & \text{иначе} \end{cases}$$

Порог 128 (ровно 50% плотности текстуры) надежно изолирует твердое ядро дорожного полотна, отсекая грязевые брызги и полупрозрачные переходы.¹

Скелетонизация (Morphological Thinning)

Для получения центральной осевой линии дороги (medial axis) шириной ровно в один пиксель применяется алгоритм морфологического истончения (Skeletonization). В экосистеме Python для этого оптимально использовать библиотеку scikit-image и функцию `skimage.morphology.skeletonize`.¹⁹

Алгоритм (часто базирующийся на методе Чжан-Суэня (Zhang-Suen) или Го-Холла (Guo-Hall)²⁰) итеративно удаляет внешние граничные пиксели объекта, строго сохраняя топологическую связность.²² Разрешение исходной матрицы 64×64 на чанк категорически нельзя уменьшать (downscale до 16×16 или 8×8). Даунскейлинг приведет к слиянию близкорасположенных параллельных троп, уничтожению узких участков дорог и формированию ложных разрывов.²² Алгоритм скелетонизации должен работать исключительно на нативном массиве 1024×1024, скомпонованном из оригинальных альфа-карт.²³

Векторизация и Прунинг графа (Graph Extraction and Pruning)

Полученный однопиксельный скелет представляет собой матрицу, которую необходимо конвертировать в математический граф.

1. **Детекция узлов (Node Detection):** В бинарном скелете любой пиксель, имеющий более двух соседей по 8-связной окрестности, классифицируется как перекресток (графный узел). Пиксели с одним соседом помечаются как тупики.
2. **Трассировка ребер (Edge Tracing):** От каждого узла иницируется обход соседних "дорожных" пикселей до столкновения со следующим узлом. Сформированный массив точек составляет геометрию ребра (сплайн).
3. **Прунинг (Удаление шпор):** Фундаментальным недостатком морфологического истончения является образование коротких ложных ответвлений ("шпор" или "заусенцев") на неровных краях растровой дороги.²⁵ Ребра, оканчивающиеся тупиком и имеющие длину менее порогового значения (например, 5-8 ярдов), алгоритмически удаляются из графа.²⁶
4. **Аппроксимация:** Для уменьшения количества избыточных точек на длинных прямых перегонах к ребрам применяется вероятностное преобразование Хафа (Probabilistic Hough Transform)²⁶ или алгоритм Рамера-Дугласа-Пекера, что превращает попиксельный путь в оптимизированный набор (X,Y) вейпойнтов.

Технический инструментарий: Экосистема Python для парсинга WoW

Создание описанного конвейера извлечения требует надежного инструментария для работы с проприетарными форматами Blizzard. Среда Python обладает наиболее зрелой

экосистемой для решения этих задач.

Интеграция с MPQ архивами

Для извлечения ADT и BLP файлов напрямую из клиентских архивов Data\common.MPQ и Data\patch.MPQ используются две основные библиотеки:

1. `mpyq` — это библиотека, написанная на чистом Python.²⁷ Она обладает выдающейся кроссплатформенностью и простотой интеграции, так как не требует компиляции C-расширений. Однако при необходимости массового извлечения сотен мегабайт ADT файлов скорость ее работы может стать узким местом.
2. `stormlib-python` (`pyStormLib`) — биндинг к классической C++ библиотеке `StormLib`.³ Она обеспечивает нативную производительность, автоматическую склейку цепочек патчей (patch archives) и поддержку зашифрованных MPQ (что не актуально для 3.3.5a, но полезно для универсальности).²

Декодирование формата BLP2

Для визуальной верификации дорожных текстур необходимо конвертировать извлеченные .blp файлы в стандартные форматы (PNG, JPEG). Стандартом де-факто в Python является библиотека `Pillow` (PIL). Хотя `Pillow` "из коробки" не имеет прямой поддержки BLP, сообщество поддерживает плагин `BlpImagePlugin.py`.⁸

Этот плагин позволяет `Pillow` прозрачно декодировать все три формата сжатия, применяемых в BLP2 (WotLK):

- `Paletted` (`compression=1`): Индексированные цвета с палитрой.
- `DXT1/DXT3/DXT5` (`compression=2`): Блочное сжатие (S3TC), которое плагин распаковывает в полноценный BGRA массив.⁸
- `Uncompressed BGRA` (`compression=3`): Сырые пиксельные данные.⁸ Для интеграции достаточно поместить `BlpImagePlugin.py` в рабочую директорию скрипта и импортировать его до вызова `Image.open()`, после чего `Pillow` автоматически зарегистрирует формат в своих внутренних дешифраторах.²⁹

Парсинг ADT файлов: Готовые решения

Вместо написания собственного парсера бинарных структур через стандартный модуль `struct` (что сопряжено с высоким риском ошибок из-за обратных тегов и динамического размера MCAL), рекомендуется опираться на существующие open-source проекты.

- `pywowlib`: Библиотека на чистом Python 3.5+, специально спроектированная для чтения форматов WotLK (3.3.5a).³¹ Она предоставляет высокоуровневые объектно-ориентированные абстракции для работы с чанками MCNK, MCLY и MCAL, избавляя от необходимости ручного вычисления `offsetInMCAL` и управления `Endianness`.

- wow-adt: Современная высокопроизводительная библиотека, написанная на Rust (с возможностью создания Python-биндингов через PyO3).⁵ Ее ключевое преимущество — двухпроходная архитектура (Two-pass parser). На первом проходе (Discovery) она мгновенно читает MCIN и маппит смещения чанков, а на втором (Parse) извлекает данные только запрошенных блоков (например, исключительно MCAL и MCLY), игнорируя тяжеловесные данные высот MCVT и геометрии объектов. Это позволяет многократно ускорить пакетную обработку континентов.⁵

Заключение

Восстановление топологии дорожных сетей в клиенте World of Warcraft 3.3.5a требует глубокого синтеза реверс-инжиниринга бинарных структур и алгоритмической геометрии. Дороги, будучи лишь процедурными альфа-масками (texture splatting), лишены явного математического описания в клиентских файлах.

Успешная реализация алгоритма базируется на строгой последовательности:

1. Использование абсолютных смещений MCIN и относительных смещений в заголовках MCNK для навигации.¹
2. Семантическая идентификация дорожных слоев в MCLY через привязку их effectId к GroundEffectTexture.dbc, где маркер массива Doodad, равный -1, служит безошибочным признаком утопанной поверхности.¹¹
3. Адаптивное декодирование альфа-масок MCAL (учитывая флаги сжатия 0x200 и разрядность MPHD).¹
4. Точная математическая трансляция пикселей 64×64 в глобальные мировые координаты на основе позиций Северо-Западных углов чанков.¹
5. Применение методов морфологической скелетонизации в нативном разрешении для извлечения графовых осей с последующим удалением топологических артефактов.¹⁹

Комбинирование описанной теоретической базы с библиотеками stormlib-python, Pillow и специализированными парсерами вроде ruwowlib обеспечивает надежный фундамент для построения утилиты, способной автономно генерировать высокоточные навигационные меши для любых нужд.

Works cited

1. ADT/v18 - wowdev, accessed February 25, 2026, <https://wowdev.wiki/ADT/v18>
2. wow-mpq — Rust parser // Lib.rs, accessed February 25, 2026, <https://lib.rs/crates/wow-mpq>
3. MPQ Archives - StormLib - Zezula.net, accessed February 25, 2026, <http://www.zezula.net/en/mpq/stormlib/sfileopenarchive.html>
4. ADT File - ClientFiles - getMaNGOS | The home of MaNGOS, accessed February 25, 2026, <https://www.getmangos.eu/wiki/referenceinfo/clientfiles/adt-file-r20028/>

5. wow_adt - Rust - Docs.rs, accessed February 25, 2026, <https://docs.rs/wow-adt>
6. wow-adt - crates.io: Rust Package Registry, accessed February 25, 2026, <https://crates.io/crates/wow-adt>
7. Alpha - wowdev, accessed February 25, 2026, <https://wowdev.wiki/Alpha>
8. Pillow/src/PIL/BlpImagePlugin.py at main - GitHub, accessed February 25, 2026, <https://github.com/python-pillow/Pillow/blob/main/src/PIL/BlpImagePlugin.py>
9. Elwynn Forest ALL Quests | WoW Classic: Quest Guide Video - YouTube, accessed February 25, 2026, <https://www.youtube.com/watch?v=wwRsUWOURvg>
10. Elwynn Forest - Wowpedia - Your wiki guide to the World of Warcraft, accessed February 25, 2026, https://wowpedia.fandom.com/wiki/Elwynn_Forest
11. DB/GroundEffectTexture - wowdev, accessed February 25, 2026, <https://wowdev.wiki/DB/GroundEffectTexture>
12. Noggit: Alphamapping & 4 Layer System | WoW - Development #1 [3.3.5a] - YouTube, accessed February 25, 2026, https://www.youtube.com/watch?v=bjluoF_CwEY
13. Seg-Road: A Segmentation Network for Road Extraction Based on Transformer and CNN with Connectivity Structures - MDPI, accessed February 25, 2026, <https://www.mdpi.com/2072-4292/15/6/1602>
14. WDT - wowdev, accessed February 25, 2026, <https://wowdev.wiki/WDT>
15. ADT/v23 - wowdev, accessed February 25, 2026, <https://wowdev.wiki/ADT/v23>
16. Path Planning in Narrow Road Scenarios Based on Four-Layer Network Cost Structure Map, accessed February 25, 2026, <https://www.mdpi.com/1424-8220/25/9/2786>
17. Improved A* Path Planning Method Based on the Grid Map - PMC, accessed February 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9416044/>
18. Research on Urban Road Network Extraction Based on Web Map API Hierarchical Rasterization and Improved Thinning Algorithm - MDPI, accessed February 25, 2026, <https://www.mdpi.com/2071-1050/14/21/14363>
19. skimage.morphology — skimage 0.26.0 documentation, accessed February 25, 2026, <https://scikit-image.org/docs/stable/api/skimorphology.html>
20. preslavsh/skeletonization - GitHub, accessed February 25, 2026, <https://github.com/preslavsh/skeletonization>
21. thinning_py3 · PyPI, accessed February 25, 2026, https://pypi.org/project/thinning_py3/
22. ROAD NETWORK EXTRACTION USING GPS TRAJECTORIES BASED ON MORPHOLOGICAL AND SKELETONIZATION ALGORITHMS, accessed February 25, 2026, <https://isprs-archives.copernicus.org/articles/XLIII-B4-2022/239/2022/>
23. skeletonization via distance maps and zero sets - Alfred Bruckstein, accessed February 25, 2026, <https://freddy.cs.technion.ac.il/wp-content/uploads/2018/01/skeletonization-via-distance-maps-and-level-sets.pdf>
24. A New, Rapid, Automated Grid Stitching Algorithm, accessed February 25, 2026, https://sbgf.org.br/mysbgf/eventos/expanded_abstracts/6th_CISBGf/Potential_Field_Methods_Methodology/sbgf384.pdf
25. nielsvogell/road-extraction: Python tools to extract the road network from

- satellite images. (class project) - GitHub, accessed February 25, 2026,
<https://github.com/nielsvogell/road-extraction>
26. Skimage skeletonize or OpenCV implementation that respects a grid? - Stack Overflow, accessed February 25, 2026,
<https://stackoverflow.com/questions/66995948/skimage-skeletonize-or-opencv-implementation-that-respects-a-grid>
27. eagleflo/mpyq: Python library for reading MPQ archives. - GitHub, accessed February 25, 2026, <https://github.com/eagleflo/mpyq>
28. vjeux/pyStormLib: Python StormLib Wrapper - GitHub, accessed February 25, 2026, <https://github.com/vjeux/pyStormLib>
29. Writing your own image plugin documentation missing information #4818 - GitHub, accessed February 25, 2026,
<https://github.com/python-pillow/Pillow/issues/4818>
30. Writing your own image plugin - Pillow (PIL Fork) 12.1.1 documentation, accessed February 25, 2026,
<https://pillow.readthedocs.io/en/stable/handbook/writing-your-own-image-plugin.html>
31. wowdev/pywowlib: Python library featuring reading and writing of various World of Warcraft files - GitHub, accessed February 25, 2026,
<https://github.com/wowdev/pywowlib>
32. Community Projects - wowdev, accessed February 25, 2026,
https://wowdev.wiki/Community_Projects